

21-Feb-2026

Date
Page

Object Oriented Javascript

- Dom's ek alag speciality hai javascript ka vo core language ka part hi hai hai

Interview topics

- this()
 - call()
 - bind()
 - apply()
 - new()
 - proto type()
- Interviews make on break hai ye topics

12 - this

console.log(this);

ye ek global context hai
par global bhi kha pe hai
node pe ki browser pe

- agar node pe hoga to empty object {}
- or agar window object me matlab browser me to undefined

ex.

```
function ranveerOnGlobalStage() {  
    return typeof this;  
}
```

Object

- this() keyword refers to the context where a piece of code is supposed to run, function or object.

Note:- this kisko point ka raha hai pehle vo samjao or this kam hai

interview question

```
function ranveerWithNoScript () {
```

```
  "Use strict";
```

```
  return this;
```

```
}
```

```
console.log(ranveerWithNoScript());
```

→ undefined

Agar ye na use kiya to reference dega global object ka, ye node ka global object hai (No-strict)

Agar ye browser me run kiya to window object ka reference dega.

browser ka global object window hai.

In Objects

```
const bollywoodfilm = {
```

```
  name: "Bajirao Mastani",
```

```
  lead: "Ranveer",
```

```
  introduce () {
```

```
    return `${this.lead} performs in ${this.name}`;
```

```
  };
```

```
}
```

```
console.log(bollywoodfilm.introduce());
```

output:-

→ Ranveer performs in Bajirao Mastani

Note:- arrow function ke pass this ka reference nahi hota hai

Date _____

Page _____

Interview question

```
const FilmDirector = {  
  name: "Sanjay Leela Bhansali",  
  cast: ["Raveer", "Deepika", "Priyanka"],  
  announceCast() {
```

```
    this.cast.forEach((actor) => {  
      console.log(`${this.name} introduces ${actor}`);  
    });  
  }  
};  
FilmDirector.announceCast();
```

Short
hand
notation

```
→ this.cast.forEach((actor) => {  
  console.log(`${this.name} introduces ${actor}`);  
});  
};  
};
```

FilmDirector.announceCast();

Code analyse

- FilmDirector object hai jisme name aur cast hai
- name ek string hai, object ki ek property hai
- cast ek array hai, object ki property hai
- announceCast ek regular function hai or uske pass this ka access hai
- this.cast se cast ko access kar sakte hai
- this.cast.forEach loop lagaya hai or iske pass access hai
- to jo arrow function (=>) hai jiske pass this ka access nahi hai yaha yeha kuchh hoga hi nahi
- arrow function sirf confined karne ko diya gaya hai

Output

Sanjay Leela Bhansali introduces Raveer

Sanjay Leela Bhansali introduces Deepika

Sanjay Leela Bhansali introduces Priyanka

Note:- default strict mode in chalta hai

Date _____
Page _____

ex.

```
const filmSet = {  
  crew: "spot boys",  
  prepareProps () {  
    console.log('Outer this.crew: ' + this.crew);  
  
    function arrangeChairs () {  
      console.log('Inner this.crew: ' + this.crew);  
    }  
  
    arrangeChairs();  
  },  
};  
filmSet.prepareProps();
```

code analyse

- crew ek string hai jiske value spot boys hai
- prepareProps function allow hai or uske pass this ka bhi access hai
or this current object ko hi to point kar raha hai
- prepareProps function ke under jo function arrangeChairs hai uske pass this ka access nahi hai
- Kyu ki A regular nested function does not inherit this isliye vo undefined hoga

Output

Outer this.crew: spot boys

Inner this.crew: undefined

Note:- Arrow functions ke pass this ka access nahi hota
aur kuchh case me hota hai vo javascript ka
behaviour hai.

ex

```
const filmset = {
  crew: "Spot boys",
  prepareProps() {
    console.log('Outer this crew: ' + this.crew);
  }
}
```

```
function arrangeChairs() {
  console.log('Inner this crew: ' + this.crew);
}
arrangeChairs();
```

```
const arrangeLights = () => {
  console.log('Arrow this crew: ' + this.crew);
};
```

```
arrangeLights();
};
```

```
Filmset.prepareProps();
```

Output:-

Outer this crew: Spot boys

Inner this crew: undefined

Arrow this crew: spot boys

language ka behaviour, hai

- Function ke under ~~fun~~ regular function ke pass this ka access nahi hota hai, vo this ko inherit nahi karta.

- par function ke under arrow function ke pass this ka access hota hai, vo this ko inherit karta hai

Note:- Javascript ke pass kuchh main apni property or methods hote hai, jese ki `dataTypes`, `new`, `this`, apply note hai, par ensabhe kuchh chize attached ki jati hai, jese, `timer`, `Dom`, `fetch` ye chare javascript ^{Date} ^{Page} ~~part~~ nahi hai, or iske pass web request ye bhi nahi hote hai

Detached Methods

- Jab tum kisi object ke method ko unke original context yani unke object se nikal kar kisi variable me store kar lete ho ya kisi callback function ki tarh pass kar dete ho, use hi detached method kehte hai

ex,

```
const actor = {  
  name: "Ranveer",  
  bow() {  
    return `${this.name} takes a bow`  
  }  
}
```

```
const detachedBow = actor.bow;
```

→ function reference

```
console.log(detachedBow());
```

Output

undefined takes a bow

Note:- (yad rakho)

- this kaha run ho raha hai, browser me run hoga to window object, agar node environment me to global object.
- `typeof` this object hota hai, `undefined`.
- 'useStrict' se undefined value dega.
- regular function ke under this ka reference hota hai.
- function ke under regular function ke pass this ka reference nahi hota hai.
- function ke under arrow function ke pass this ka reference nahi hota hai.
- detached method ke pass bhi this ka reference nahi hota hai.

Note:- Closure ke ander reference ko hold kar sakte hai or detached method me reference nahi milta hai.

13 - call(), bind(), apply()

ex.

1) call and apply $\Rightarrow$ basic chef

(Kitchen vegem sara saman aapko

dena padta hai) - immediately cook kar deta hai

2) bind $\Rightarrow$ always returns a new function.

(uske team ke koi to aayega hi)

Note:- Her methods ke pass apne call(), bind(), apply() hote hi hai

1) call()

- Immediately invokes the function and accepts arguments individually in a comma-separated list.

ex.

```
function (cookDish <ingredient, style>) {  
  return `${this.name} prepare ${ingredient} in  
  ${style} style!`  
}
```

name to define kina hai
to yese bahar

$\rightarrow$ const sharmaKitchen = { name: "Sharma jis kitchen" }

$\rightarrow$ const guptaKitchen = { name: "Gupta jis kitchen" }

```
console.log(cookDish.call(sharmaKitchen, "Paneer and  
spices", "Mughlai"))
```

argument ki list

output

Sharma jis Kitchen prepares Paneer and spices
in Mughlai style!

Note:- call() ke subke liye use karo par array
ke liye hamesh apply() use karo.

Note:- apply or call returns result.
But bind returns the function.
- call(), bind(), apply() modify the this

2) apply() [array $\Rightarrow$ array]

Immediately invokes the function and accepts arguments as a single array

ex,

```
const guptaOrder = ["chole kulche", "Punjabhi dhaba"]  
console.log(cookDish.apply(guptaKitchen, guptaOrder));
```

output

Gupta jis Kitchen prepares chole kulche in ~~flat~~
Punjabhi dhaba style!

ex

```
const bills = [100, 30, 45, 50]
```

Math.max.apply(null, bills) $\Rightarrow$ 100

Math.max(...bills) $\Rightarrow$ 100

↳ Kyki pehle argument dena jaruri hai : to null likhna vaha technical requirement hai

3) bind()

- It returns a new function with the specified this value

example of call(), bind() & apply()

```
function reportDelivery(location, status) {  
  return ` ${this.name} at ${location}: ${status}`;  
}
```


const deliveryBoy = { name: "Ranveer" }

console.log("Call:", reportDelivery.call(deliveryBoy, "Lyari", "Ordered"))

console.log("Apply:", reportDelivery.apply(deliveryBoy, ["Mans", "Pick up"]))

→ console.log("Bind:", reportDelivery.bind(deliveryBoy, "Haridwar", "What"));

bind doesn't work this way
it gives [function: bound reportDelivery]

the correct way to write bind()

1st hold in variable

const bindReport = reportDelivery.bind(deliveryBoy, "Haridwar", "What");

console.log(bindReport());

or (Both are same)

const bindReport = reportDelivery.bind(deliveryBoy);

console.log(bindReport("Haridwar", "What"));

Output

Call: Ranveer at Lyari: Ordered

Apply: Ranveer at Mans: Pick up

Ranveer at Haridwar: What

- Function Constructors are used to create and initialise objects with specific properties and method.

ex, this is method
modelName is property

Date _____
Page _____

14- new()

- The new keyword is used to create an instance of an object that has a constructor function.

ex,

```
function TataCar(chassisNumber, modelName) {  
  this.chassisNumber = chassisNumber;  
  this.modelName = modelName;  
  this.fuelLevel = 100;  
}
```

```
TataCar.prototype.status = function() {  
  return `Tata ${this.modelName} # ${this.chassisNumber} | fuel: ${this.fuelLevel}`;  
};
```

```
const car1 = new TataCar("MH-101", "Nexon");  
const car2 = new TataCar("DL-202", "Harrier");
```

```
console.log(car2.modelName);  
console.log(car2.modelName);  
console.log(car1.status());  
console.log(car2.status());
```

output:

Tata Nexon #MH-101 | fuel:100

Tata Harrier #DL-202 | fuel:100

- 1st - Jese hi aapne new keyword lagaya wese hi aapke liye ek empty object {} banaya jata hai
- 2nd - Is empty object ka prototype or TataCar ke prototype ko link kiya jata hai
- 3rd - Ab ye this keyword activate hota hai, jo bhi ise call kar raha hota hai uske sat ye bind karta hai
this matlab un variable ka reference
- jese car1 jo value provide kar raha hai vo this ke under dali jati hai
- or isi tarah car2 ko bhi apna this milta hai
- 4th - agar constructor explicitly ko object return nahi karta hai to ye kam new ka hota hai.

Note:- function ya classes se dono me new keyword same hota hai, object ke under bhi new create new intences.

// This is not same as above

```
function createAutoRickshaw (id, route) {
    return {
        id,
        route,
        run() {
            return `Auto ${this.id} running on ${this.route}`;
        },
    };
}
```

Factory
Function

const auto1 = createAutoRickshaw("UP-1", "Lucknow-Kanpur");

→ yaha jitne bhi variable ka reference hoga vo same auto1 or auto2 ke pass jayenge

Const auto2 = createAutoRickshaw ("UP-2, "Agra - Mathura");

Console.log (auto1.run());

Console.log (auto2.run());

Output

Auto UP-1 running on Lucknow - Kanpur

Auto UP-2 running on Agra - Mathura

→ Each instance getting is own copy of method

- Factory functions (reference ko apne sat leke chalta hai) if any function that creates and returns a new object without the use of the new keyword

- Constructor ke under new keyword involve hota hai, or jab aap constructor use karte ho tab aap shared memory use karte ho.

- Or Factory function me par instance hota hai matlab jese auto1 ek hai or auto2

15- Prototype

In every object has an internal link to another object is called prototype.

and the important use is inheritance.

- Object.create() - this method is used to create new object, with specified prototype object and optional properties.

- or ye sabke pass hoti hai

en property ko add or set bhi kar sakte hai

ex,

```
const prithviraj = {  
  name: "Prithviraj",  
  generation: "grandfather",  
  cookTraditionalDish() {  
    return `$$ this.name } cooks an ancient  
    family recipe`;  
  }  
};
```

```
const raj = Object.create(prithviraj);  
raj.name = "raj";  
raj.generation = "father";  
raj.business = function() {  
  return `$$ this.name } runs the family  
  business`;  
}
```

```
console.log(raj);
```

```
const ranbir = Object.create(raj)  
ranbir.name = "ranbir";  
ranbir.generation = "son";  
ranbir.makemakefilm = function() {  
  return `$$ this.name } directs blockbuster  
  movies`;  
}
```

```
console.log(ranbir.makefilm());
```

```
console.log(ranbir.ranbusiness());
```

```
console.log(ranbir.makefilm());
```


Note:- this ka matlab jo call karre

Date _____
Page _____

Output

ranbir directs blockbuster movies

ranbir runs the family business

ranbir cooks the ~~traditional~~ ancient family recipe

- Prototype ke power jo string or array, dono function sub ke pass hai

```
Array.prototype.last = function () {  
    return this[this.length - 1];  
};
```

```
console.log([1,2,3].last());
```

⇒ 3

```
console.log(["Kaka", "Maka", "Dipali"].last().last());
```

⇒ Dipali

- Polyfill

- A polyfill is a piece of code used to provide modern functionality on older browsers that do not natively support it.

ex.

```
Array.prototype.me = "me";
```

```
console.log([1,2,3].me)
```